



UANL

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN



FIME

FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN
FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

“Tarea Práctica #4: Monitoreo estático (Equipo)”

Sistemas Distribuidos y Paralelos

M.I.I. Carlos Adrián Pérez Cortez

Grupo: 004

Hora: N5

Salón: 4207

Equipo 9

MATRICULA	NOMBRE	PE
1952947	Heidi Pamela Martínez Martínez	ITS
1845517	Andrea Monserrat Méndez Flores	ITS
2109147	Alexia Hernandez Andrade	ITS
2004428	Alejandro Sauqué Pérez	ITS
2077884	Melida Krystal Mendoza Ventura	ITS

Ciudad Universitaria, San Nicolás de los Garza, Nuevo León a 23 de octubre
de 2024

Contenido

OBJETIVO	3
DIFICULTADES ENCONTRADAS	3
CAPTURA DE PANTALLA DEL CÓDIGO EN JAVA (CLIENTE Y SERVIDOR)	4
CAPTURA DE PANTALLA DE LOS RESULTADOS OBTENIDOS EN CONSOLA (CLIENTE Y SERVIDOR)	7
Conclusiones	10

OBJETIVO

El objetivo de este proyecto es crear un cliente y un servidor en Java que se comuniquen a través de sockets. El cliente debe obtener y mostrar sus propias características del sistema, como el nombre de la máquina, el sistema operativo, el modelo del procesador, el número de núcleos y el tamaño del disco duro. El servidor debe estar preparado para recibir esta información del cliente.

DIFICULTADES ENCONTRADAS

Nos hemos enfrentado a varias dificultades en el desarrollo. Uno de los principales retos ha sido el manejo adecuado de las excepciones, especialmente las relacionadas con entrada/salida (I/O). Capturar y gestionar estas excepciones es fundamental para diagnosticar problemas en la conexión o en la transmisión de datos, lo cual ha requerido atención detallada para no pasar por alto errores críticos.

Otro desafío importante ha sido la serialización de los objetos que se envían entre el cliente y el servidor. Hemos tenido que asegurarnos de que los datos se serialicen y deserialicen correctamente, ya que cualquier discrepancia en el tipo de datos puede generar excepciones y detener la ejecución del programa.

En cuanto a la visualización de datos, implementar una interfaz gráfica que muestre las características del cliente no ha sido tan sencillo como esperaba. Actualizar la tabla en la interfaz de manera dinámica con la información recibida ha requerido ajustes en la lógica de la interfaz de usuario y ha sido un aspecto clave en el funcionamiento final del proyecto, teniendo como problema el que el cliente no viera sus datos sino los del servidor, o que los viera correctos, pero no se enviaran al servidor.

Las pruebas de comunicación entre el cliente y el servidor fueron presentes dificultades. Verificar que los mensajes se envían y reciben correctamente ha demandado múltiples intentos y ajustes, ya que algunos problemas han surgido en la lógica de cómo se gestionan las solicitudes y respuestas.

Finalmente, la configuración del entorno ha sido otro aspecto desafiante, asegurarnos de que el entorno de desarrollo esté configurado correctamente, con todas las bibliotecas y dependencias en su lugar, ha sido fundamental para que el proyecto funcione sin problemas. Además, la sincronización entre el cliente y el servidor, asegurando que no haya bloqueos ni respuestas indeseadas, ha sido un detalle en el que hemos tenido que enfocarnos para optimizar la comunicación entre ambos.

CAPTURA DE PANTALLA DEL CÓDIGO EN JAVA (CLIENTE Y SERVIDOR)

SERVIDOR

```
1 package libro;
2
3 import javax.swing.*;
4 import javax.swing.table.DefaultTableModel;
5 import java.io.*;
6 import java.net.*;
7
8 public class Server {
9     public static void main(String[] args) {
10         int port = 5355; // Puerto de escucha del servidor
11
12         JFrame serverFrame = new JFrame("Servidor");
13         serverFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
14
15         // Crear una tabla para mostrar las características
16         DefaultTableModel tableModel = new DefaultTableModel();
17         JTable table = new JTable(tableModel);
18         tableModel.addColumn("Característica");
19         tableModel.addColumn("Valor");
20
21         serverFrame.add(new JScrollPane(table));
22         serverFrame.pack();
23         serverFrame.setVisible(true);
24
25         // Mostrar las características del servidor al inicio
26         String[] serverFeatures = getSystemInfo();
27         for (String feature : serverFeatures) {
28             String[] parts = feature.split(":");
29             addRowToTable(tableModel, parts[0], parts[1]);
30         }
31
32         try (ServerSocket serverSocket = new ServerSocket(port)) {
33             System.out.println("Esperando conexiones en el puerto " + port);
34
35             while (true) {
36                 Socket clientSocket = serverSocket.accept();
37                 System.out.println("Cliente conectado desde: " + clientSocket.getInetAddress());
38
39                 // Obtener entrada y salida de datos
40                 ObjectOutputStream outputStream = new ObjectOutputStream(clientSocket.getOutputStream());
41                 ObjectInputStream inputStream = new ObjectInputStream(clientSocket.getInputStream());
42
43                 // Recibir solicitud del cliente para obtener características
44                 Object request = inputStream.readObject();
45                 if (request instanceof String && ((String) request).equals("request")) {
46                     // Obtener y enviar características del servidor al cliente
47                     outputStream.writeObject(serverFeatures);
48                 }
49
50                 // Recibir características del cliente
51                 Object clientResponse = inputStream.readObject();
52                 if (clientResponse instanceof String[]) {
53                     String[] clientFeatures = (String[]) clientResponse;
54                     for (String feature : clientFeatures) {
55                         String[] parts = feature.split(":");
56                         addRowToTable(tableModel, "Cliente " + parts[0], parts[1]);
57                     }
58                 }
59
60                 // Cierra la conexión con el cliente
61                 clientSocket.close();
62                 System.out.println("Conexión cerrada con el cliente.");
63             }
64         } catch (Exception e) {
65             e.printStackTrace();
66         }
67     }
68
69     // Método para obtener las características del sistema
70     private static String[] getSystemInfo() {
71         String[] features = new String[4]; // Tamaño del arreglo ajustado para las nuevas características (sin velocidad
```

```

71 String[] features = new String[4]; // Tamaño del arreglo ajustado para las nuevas características (sin version)
72 try {
73     // Información del sistema operativo
74     String osName = System.getProperty("os.name");
75     String osVersion = System.getProperty("os.version");
76     features[0] = "Versión del sistema operativo: " + osName + " " + osVersion;
77
78     // Información del hardware
79     String processorModel = System.getenv("PROCESSOR_IDENTIFIER");
80     String processorCores = System.getenv("NUMBER_OF_PROCESSORS");
81     features[1] = "Modelo del procesador: " + processorModel;
82     features[2] = "Número de núcleos: " + processorCores;
83
84     // Tamaño del disco duro
85     File file = new File("/");
86     long totalDiskSpace = file.getTotalSpace();
87     features[3] = "Tamaño del disco duro: " + totalDiskSpace + " bytes";
88
89 } catch (Exception e) {
90     e.printStackTrace();
91 }
92 return features;
93 }
94
95 // Método para agregar una fila a la tabla
96 private static void addRowToTable(DefaultTableModel tableModel, String feature, String value) {
97     tableModel.addRow(new String[]{feature, value});
98 }
99 }
100

```

CLIENTE

```

1 package libro;
2
3 import javax.swing.*;
4 import javax.swing.table.DefaultTableModel;
5 import java.io.*;
6 import java.net.*;
7
8 public class Cliente {
9     public static void main(String[] args) {
10         String serverAddress = "25.47.153.25"; // Dirección IP del servidor
11         int port = 5355; // Puerto del servidor
12
13         // Crear una ventana para mostrar las características del cliente
14         JFrame clientFrame = new JFrame("Cliente");
15         clientFrame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
16
17         // Crear una tabla para mostrar las características
18         DefaultTableModel tableModel = new DefaultTableModel();
19         JTable table = new JTable(tableModel);
20         tableModel.addColumn("Característica");
21         tableModel.addColumn("Valor");
22
23         clientFrame.add(new JScrollPane(table));
24         clientFrame.pack();
25         clientFrame.setVisible(true);
26
27         try {
28             Socket socket = new Socket(serverAddress, port);
29
30             // Obtener entrada y salida de datos
31             ObjectOutputStream outputStream = new ObjectOutputStream(socket.getOutputStream());
32             ObjectInputStream inputStream = new ObjectInputStream(socket.getInputStream());
33
34             // Enviar solicitud al servidor para obtener características
35             outputStream.writeObject("request");
36

```

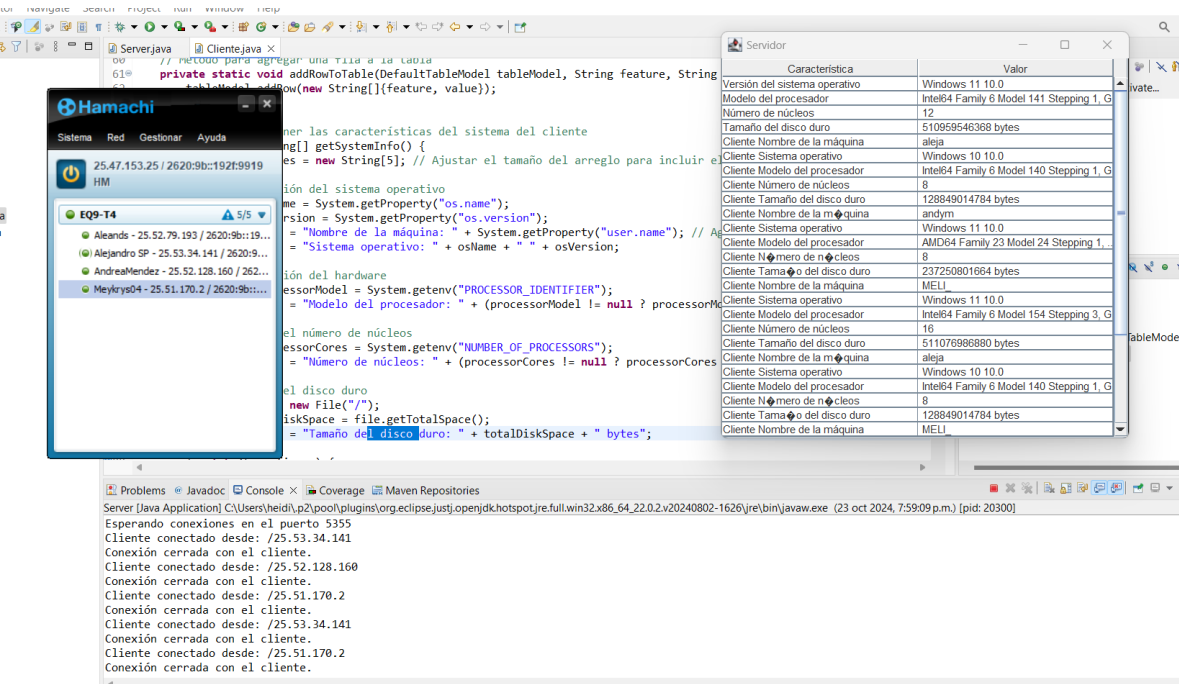
```

37 // Obtener y enviar características del cliente al servidor
38 String[] clientFeatures = getSystemInfo();
39 outputStream.writeObject(clientFeatures);
40
41 // Recibir características del servidor
42 Object response = inputStream.readObject();
43 if (response instanceof String[]) {
44     // No mostrar características del servidor
45 }
46
47 // Mostrar características del cliente en la tabla
48 for (String feature : clientFeatures) {
49     String[] parts = feature.split(":");
50     addRowToTable(tableModel, parts[0], parts.length > 1 ? parts[1] : "No disponible");
51 }
52
53 // Cierra la conexión con el servidor
54 socket.close();
55 } catch (Exception e) {
56     e.printStackTrace();
57 }
58 }
59
60 // Método para agregar una fila a la tabla
61 private static void addRowToTable(DefaultTableModel tableModel, String feature, String value) {
62     tableModel.addRow(new String[]{feature, value});
63 }
64
65 // Método para obtener las características del sistema del cliente
66 private static String[] getSystemInfo() {
67     String[] features = new String[5]; // Ajustar el tamaño del arreglo para incluir el nombre de la máquina
68     try {
69         // Información del sistema operativo
70         String osName = System.getProperty("os.name");
71         String osVersion = System.getProperty("os.version");
72
73         features[0] = "Nombre de la máquina: " + System.getProperty("user.name"); // Agregar nombre de la máquina
74         features[1] = "Sistema operativo: " + osName + " " + osVersion;
75
76         // Información del hardware
77         String processorModel = System.getenv("PROCESSOR_IDENTIFIER");
78         features[2] = "Modelo del procesador: " + (processorModel != null ? processorModel : "No disponible");
79
80         // Obtener el número de núcleos
81         String processorCores = System.getenv("NUMBER_OF_PROCESSORS");
82         features[3] = "Número de núcleos: " + (processorCores != null ? processorCores : "No disponible");
83
84         // Tamaño del disco duro
85         File file = new File("/");
86         long totalDiskSpace = file.getTotalSpace();
87         features[4] = "Tamaño del disco duro: " + totalDiskSpace + " bytes";
88
89     } catch (Exception e) {
90         e.printStackTrace();
91     }
92     return features;
93 }
94

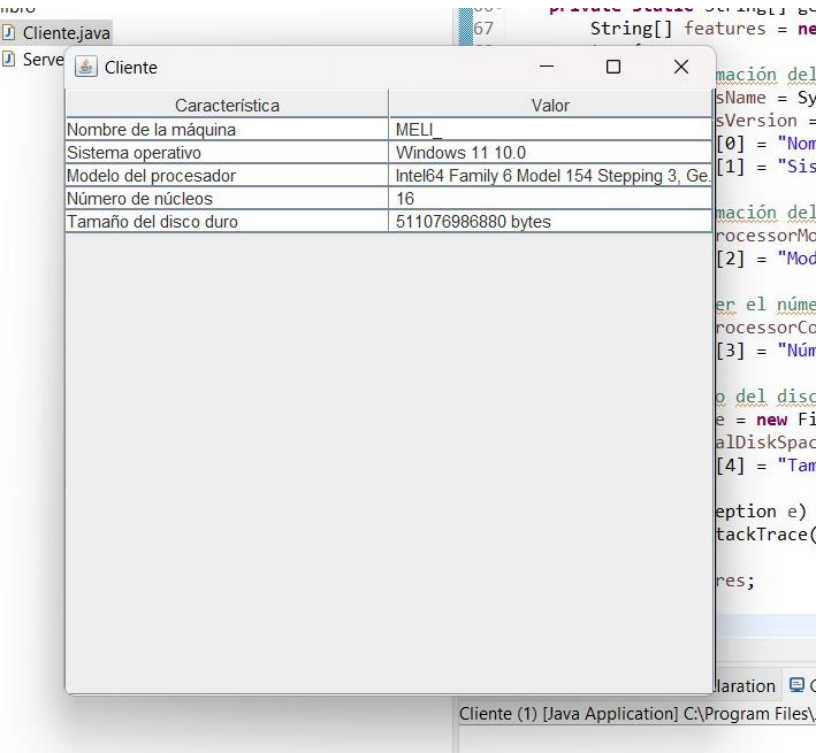
```

CAPTURA DE PANTALLA DE LOS RESULTADOS OBTENIDOS EN CONSOLA (CLIENTE Y SERVIDOR)

SERVIDOR



CLIENTE



Cliente	
Característica	Valor
Nombre de la máquina	andym
Sistema operativo	Windows 11 10.0
Modelo del procesador	AMD64 Family 23 Model 24 Stepping 1...
Número de núcleos	8
Tamaño del disco duro	237250801664 bytes

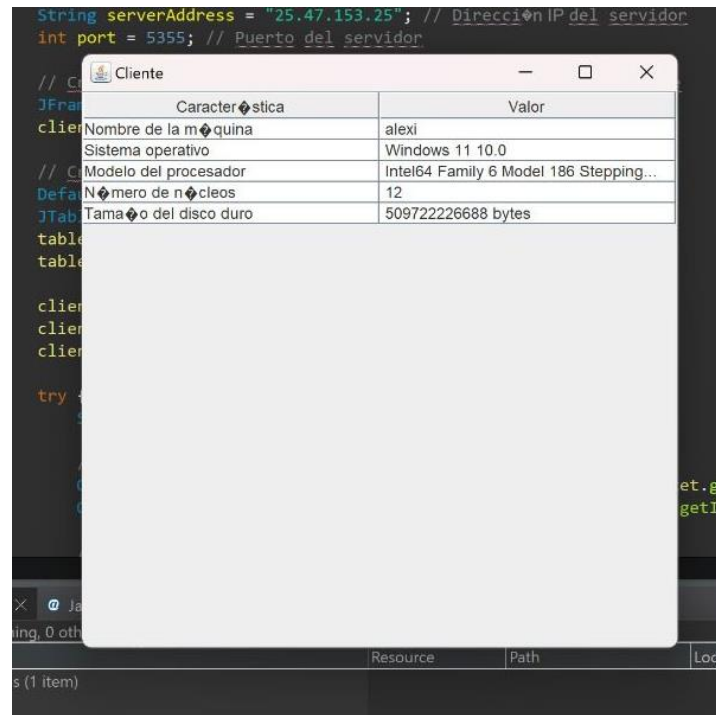
```
public class Cliente {  
    private static String[] getSystemInfo() {  
        try {  
            ;  
            property(key:"user.name", value:"aleja");  
            version;  
            IDENTIFIER");  
            el != null ? proc  
            ROCESSORS");  
            != null ? process  
        }  
    }  
}
```

Cliente	
Característica	Valor
Nombre de la máquina	aleja
Sistema operativo	Windows 10 10.0
Modelo del procesador	Intel64 Family 6 Model 140 Stepping...
Número de núcleos	8
Tamaño del disco duro	128849014784 bytes

PROBLEMS OUTPUT

C:\Users\aleja> java -cp . Cliente

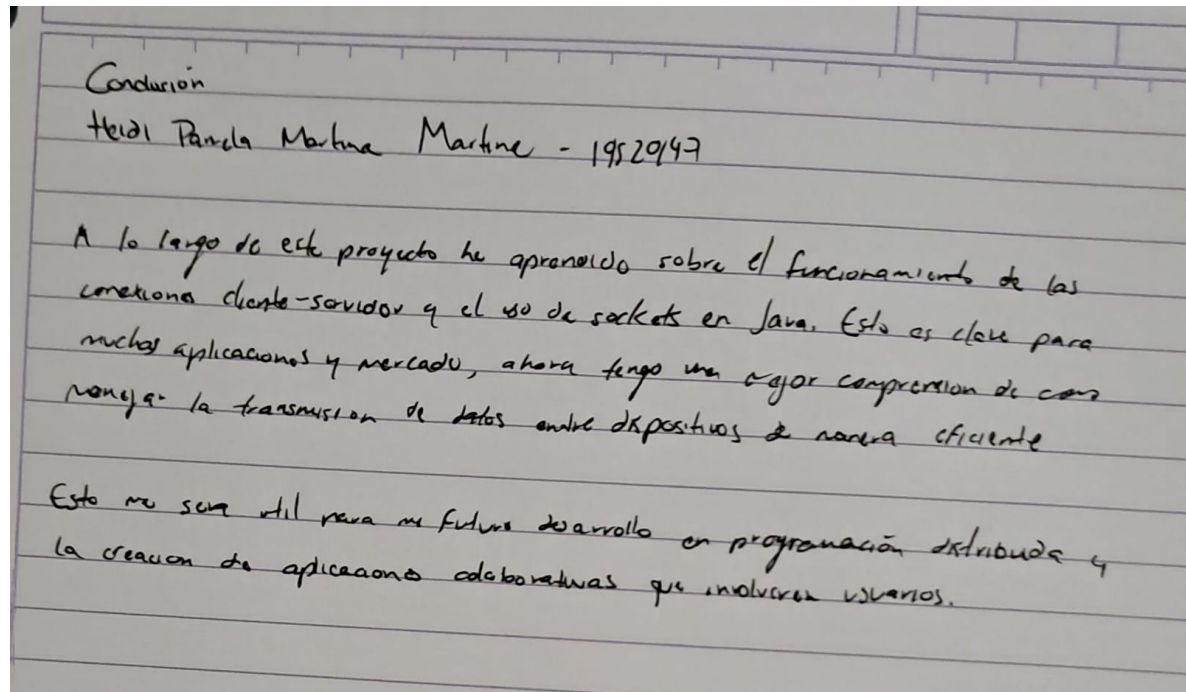
Exception in thread "main" java.lang.NoClassDefFoundError: Cliente



Cliente	
Característica	Valor
Nombre de la máquina	andym
Sistema operativo	Windows 11 10.0
Modelo del procesador	AMD64 Family 23 Model 24 Stepping 1, AuthenticAMD
Número de núcleos	8
Tamaño del disco duro	237250801664 bytes

Conclusiones

1952947 - Heidi Pamela Martínez Martínez



Andrea Monserrat Méndez Flores
1845517

Conclusión:

Esta tarea se tuvo que realizar con todo el equipo conectado a la misma red por medio de hamachi, se tuvo que leer los datos de nuestro dispositivo y mostrarlo en pantalla, al principio fue complicado, ya que al conectar el cliente, en lugar de aparecer sus propios datos, aparecían los datos del servidor, se lo debió a un error del código, una vez corregido este problema logramos que la información de cada cliente pudiera ser visualizada en su dispositivo.

Conclusión

Tarea practica #4

Alexia Hernandez Andrade 2109147

En esta actividad logramos utilizar una libreria para extraer y mostrar características clave de una computadora, utilizamos hamachi donde creamos una red virtual que nos permitio conectarnos entre sí. el servidor desarrollado en eclipse mostro la interfaz visual que recopilaba y presentaba las características de los clientes conectados, lo que facilito el monitoreo de los recursos.

Esta actividad fue muy interesante ya que manejamos una herramienta nueva, y nos llevamos grandes conocimientos por la misma.

Alejandra Sauqué Pérez

Conclusión Individual Sis. Dist. y Par.

Considero que esta práctica fue demasiado interesante debido a que tuvimos que todos encontrar una manera de estar en red para validar el funcionamiento del programa. Nosotros usamos Hamachi y personalmente creo que se pueden hacer cosas grandiasas con redes privadas. En este caso no estamos trabajando con demasiada carga de procesamiento ni en el cliente ni en el servidor. Sin embargo, considero que para usos más orientados a un sistema paralelo y distribuido útil podemos aprovechar las capacidades de cada nodo o dispositivo para lograr un objetivo / resultado particular.

